17. To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, Test Job Listings.

Code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

#define MAX_JOBS 10
#define MAX_COMMAND 100

typedef enum {
    RUNNING,
    COMPLETED
} JobState;

typedef struct {
    int job_id;
    pid_t pid;
    char command[MAX_COMMAND];
    JobState state;
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;

/* Add a job to the job table */
void addJob(pid_t pid, const char *command) {

    if (job_count >= MAX_JOBS) {
        printf("Job table is full.\n");
        return;
    }

    jobs[job_count].job_id = job_count + 1;
    jobs[job_count].pid = pid;

    strcpy(jobs[job_count].command, command);

    jobs[job_count].state = RUNNING;

    job_count++;
}

/* Update state of each job */
void updateJobStates() {

    int status;

    for (int i = 0; i < job_count; i++) {

        if (jobs[i].state == RUNNING) {

            pid_t result = waitpid(
```

```
            pid_t result = waitpid(
                jobs[i].pid,
                &status,
                WNOHANG
            );

            if (result == jobs[i].pid) {
                jobs[i].state = COMPLETED;
            }
        }
    }
}

/* List only active jobs */
void listActiveJobs() {

    updateJobStates();

    printf("\nActive Jobs\n");
    printf("-----------------------------------------\n");
    printf("JOB ID\tPID\tSTATUS\t\tCOMMAND\n");
    printf("-----------------------------------------\n");

    int found = 0;

    for (int i = 0; i < job_count; i++) {

        if (jobs[i].state == RUNNING) {

            printf("[%d]\t%d\tRUNNING\t\t%s\n",
                   jobs[i].job_id,
                   jobs[i].pid,
                   jobs[i].command);

            found = 1;
        }
    }

    if (!found) {
        printf("No active jobs.\n");
    }

    printf("-----------------------------------------\n");
}

/* Display all jobs */
void displayAllJobs() {

    updateJobStates();

    printf("\nAll Jobs\n");
    printf("-----------------------------------------\n");
```

```c
/* Display all jobs */
void displayAllJobs() {

    updateJobStates();

    printf("\nAll Jobs\n");
    printf("----------------------------------------\n");
    printf("JOB ID\tPID\tSTATUS\t\tCOMMAND\n");
    printf("----------------------------------------\n");

    for (int i = 0; i < job_count; i++) {

        if (jobs[i].state == RUNNING) {

            printf("[%d]\t%d\tRUNNING\t\t%s\n",
                   jobs[i].job_id,
                   jobs[i].pid,
                   jobs[i].command);

        } else {

            printf("[%d]\t%d\tCOMPLETED\t%s\n",
                   jobs[i].job_id,
                   jobs[i].pid,
                   jobs[i].command);
        }
    }

    printf("----------------------------------------\n");
}

/* Retrieve job ID */
void retrieveJobID(int index) {

    if (index < 0 || index >= job_count) {

        printf("Invalid job number.\n");
        return;
    }

    printf("Job ID: [%d]\n",
           jobs[index].job_id);
}

/* Create a background job */
void launchJob(const char *command, int seconds) {

    pid_t pid = fork();

    if (pid < 0) {
```

```
  GNU nano 8.7.1
void launchJob(const char *command, int seconds) {

    pid_t pid = fork();

    if (pid < 0) {

        perror("fork");
        return;
    }

    if (pid == 0) {

        /* Simulate a background task */
        sleep(seconds);

        exit(0);
    }

    /* Parent stores job information */
    addJob(pid, command);

    printf("[Job %d] Started with PID %d\n",
           jobs[job_count - 1].job_id,
           pid);
}

int main() {

    printf("Job Listing System\n");
    printf("------------------\n");

    /* Create two test jobs */
    launchJob("sleep 5", 5);
    launchJob("sleep 8", 8);

    /* Display active jobs */
    listActiveJobs();

    /* Retrieve first job ID */
    printf("\nRetrieving Job ID:\n");
    retrieveJobID(0);

    /* Wait for first job to finish */
    printf("\nWaiting 6 seconds...\n");
    sleep(6);

    /* Display updated status */
    printf("\nUpdated Job Listing:\n");
    displayAllJobs();

    /* Display only active jobs */
    printf("\nActive Job Listing:\n");
    displayAllJobs();

    /* Display only active jobs */
    printf("\nActive Job Listing:\n");
    listActiveJobs();

    /* Wait for second job */
    sleep(3);

    /* Final status */
    printf("\nFinal Job Status:\n");
    displayAllJobs();

    return 0;
}
```

Result:

```
sanjana3027@DESKTOP-QJF5AGU:        L$ nano Skill17a.c
sanjana3027@DESKTOP-QJF5AGU:        L$ gcc Skill17a.c -o s17a
sanjana3027@DESKTOP-QJF5AGU:        L$ ./s17a
--- Starting Tests ---
[Job 1] Started with PID 3056
[Job 2] Started with PID 3057

Active Jobs
-----------------------------------------
JOB ID  PID     STATUS          COMMAND
-----------------------------------------
[1]     3056    RUNNING         sleep 5
[2]     3057    RUNNING         sleep 8
-----------------------------------------

Retrieving Job ID:
Job ID: [1]

Waiting 6 seconds...

Updated Job Listing:

All Jobs
-----------------------------------------
JOB ID  PID     STATUS          COMMAND
-----------------------------------------
[1]     3056    COMPLETED       sleep 5
[2]     3057    RUNNING         sleep 8
-----------------------------------------

Active Job Listing:

Active Jobs
-----------------------------------------
JOB ID  PID     STATUS          COMMAND
-----------------------------------------
[2]     3057    RUNNING         sleep 8
-----------------------------------------

Final Job Status:

All Jobs
-----------------------------------------
JOB ID  PID     STATUS          COMMAND
-----------------------------------------
[1]     3056    COMPLETED       sleep 5
[2]     3057    COMPLETED       sleep 8
-----------------------------------------
sanjana3027@DESKTOP-QJF5AGU:        L$
```

To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching.

Code:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>
#include <string.h>
#include <termios.h>

#define MAX_JOBS 5
#define MAX_COMMAND 100

typedef enum {
    RUNNING,
    STOPPED,
    COMPLETED
} JobState;

typedef struct {
    int job_id;
    pid_t pid;
    char command[MAX_COMMAND];
    JobState state;
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;

/* Add a job */
void addJob(pid_t pid, const char *command) {

    if (job_count >= MAX_JOBS) {
        printf("Job table is full.\n");
        return;
    }

    jobs[job_count].job_id = job_count + 1;
    jobs[job_count].pid = pid;

    strcpy(jobs[job_count].command, command);

    jobs[job_count].state = RUNNING;

    job_count++;
}

/* Display jobs */
void displayJobs() {

    printf("\nJob Table\n");
    printf("-----------------------------------\n");
    printf("ID\tPID\tSTATE\t\tCOMMAND\n");
    printf("-----------------------------------\n");
```

```
 GNU nano 8.7.1
    printf("-------------------------------------\n");

    for (int i = 0; i < job_count; i++) {

        char *state;

        if (jobs[i].state == RUNNING)
            state = "RUNNING";
        else if (jobs[i].state == STOPPED)
            state = "STOPPED";
        else
            state = "COMPLETED";

        printf("[%d]\t%d\t%s\t\t%s\n",
               jobs[i].job_id,
               jobs[i].pid,
               state,
               jobs[i].command);
    }

    printf("-------------------------------------\n");
}

/* Find job by job ID */
int findJob(int job_id) {

    for (int i = 0; i < job_count; i++) {

        if (jobs[i].job_id == job_id)
            return i;
    }

    return -1;
}

/* Switch job to foreground */
void foregroundJob(int job_id) {

    int index = findJob(job_id);

    if (index == -1) {

        printf("Job [%d] not found.\n", job_id);
        return;
    }

    Job *job = &jobs[index];

    printf("\nTarget Job: [%d]\n", job->job_id);
    printf("Command: %s\n", job->command);

    /*
```

```
    /*
     * Transfer terminal control to the job.
     */
    if (tcsetpgrp(STDIN_FILENO, job->pid) == -1) {
        perror("tcsetpgrp");
    }

    printf("Terminal control transferred to job [%d].\n",
           job->job_id);

    /*
     * Resume stopped process.
     */
    if (job->state == STOPPED) {

        if (kill(job->pid, SIGCONT) == -1) {
            perror("SIGCONT");
        }

        job->state = RUNNING;

        printf("Job [%d] resumed.\n",
               job->job_id);
    }

    /*
     * Wait for foreground process.
     */
    int status;

    waitpid(job->pid, &status, WUNTRACED);

    /*
     * Update job state.
     */
    if (WIFSTOPPED(status)) {

        job->state = STOPPED;

        printf("Job [%d] stopped.\n",
               job->job_id);
    }

    else if (WIFEXITED(status) ||
             WIFSIGNALED(status)) {

        job->state = COMPLETED;

        printf("Job [%d] completed.\n",
               job->job_id);
```

```
                job->job_id);
    }

    /*
     * Give terminal back to shell.
     */
    if (tcsetpgrp(STDIN_FILENO, getpgrp()) == -1) {
        perror("tcsetpgrp");
    }

    printf("Terminal control returned to shell.\n");
}

/* Create a test foreground/background job */
void createJob() {

    pid_t pid = fork();

    if (pid < 0) {

        perror("fork");
        return;
    }

    if (pid == 0) {

        /*
         * Create a new process group.
         */
        setpgid(0, 0);

        /*
         * Simulate a job.
         */
        printf("Child process started.\n");

        sleep(5);

        printf("Child process finished.\n");

        exit(0);
    }

    /*
     * Parent creates process group.
     */
    setpgid(pid, pid);

    addJob(pid, "sleep 5");

    printf("Job [%d] created with PID %d.\n",
           jobs[job_count - 1].job_id,
```

```
    /*
     * Parent creates process group.
     */
    setpgid(pid, pid);

    addJob(pid, "sleep 5");

    printf("Job [%d] created with PID %d.\n",
           jobs[job_count - 1].job_id,
           pid);
}

int main() {

    printf("Foreground Job Switching System\n");
    printf("------------------------------\n");

    /*
     * Create test job.
     */
    createJob();

    /*
     * Give child a moment to start.
     */
    sleep(1);

    displayJobs();

    /*
     * Switch job to foreground.
     */
    printf("\nSwitching Job [1] to foreground...\n");

    foregroundJob(1);

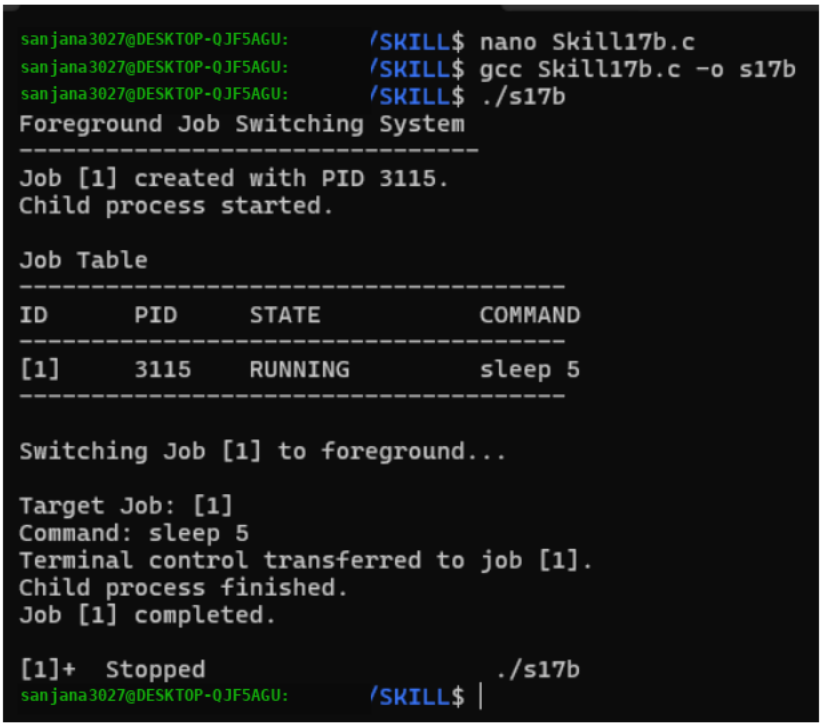
    /*
     * Display final state.
     */
    printf("\nFinal Job State:\n");

    displayJobs();

    printf("\nForeground switching completed successfully.\n");

    return 0;
}
```

Result:


sanjana3027@DESKTOP-QJF5AGU: /SKILL$ nano Skill17b.c
sanjana3027@DESKTOP-QJF5AGU: /SKILL$ gcc Skill17b.c -o s17b
sanjana3027@DESKTOP-QJF5AGU: /SKILL$ ./s17b
Foreground Job Switching System
Job [1] created with PID 3115.
Child process started.
Job Table
ID PID STATE COMMAND
[1] 3115 RUNNING sleep 5
Switching Job [1] to foreground...
Target Job: [1]
Command: sleep 5
Terminal control transferred to job [1].
Child process finished.
Job [1] completed.
[1]+ Stopped ./s17b
sanjana3027@DESKTOP-QJF5AGU: /SKILL$